

Théorie des graphes — Projet (binôme)

Graphe orienté & plus court chemin (Dijkstra) — Version finale

Objectifs

- Concevoir et coder un module Python de graphes orientés pondérés (poids réels ≥ 0).
- Implémenter l'algorithme de Dijkstra pour les plus courts chemins (poids négatifs interdits).
- Fournir une API minimale claire (francisée), des tests unitaires, une documentation et une démonstration en ligne de commande.

Contexte pédagogique

Concevoir et implémenter une solution technologique permettant de déterminer le plus court chemin entre deux positions géographiques, à partir d'un fichier CSV décrivant des arcs orientés et les coordonnées associées aux sommets.

Spécification (Exigences)

- E1. Entrée : charger un graphe depuis un fichier CSV (format défini ci-dessous).
- E2. API minimale : fournir au moins les sous-programmes suivants : ``charger_csv(fichier)``, ``plus_court_chemin(source, cible)``, éventuellement ``coordonnees_sommet(nom)``.
- E3. Chemin : ``plus_court_chemin(source, cible)`` retourne (liste_de_sommets, cout_total). Si la cible est inatteignable, retourner ([], +inf) ou traiter le problème.
- E4. Poids : les poids négatifs sont interdits. Les poids nuls sont autorisés.
- E5. Cas limites : gérer sommets absents, graphe vide, inaccessibilité, arcs de poids 0, sommets isolés, fichier graphe inexistant ou erroné.
- E6. Tests : couvrir au minimum le chargement CSV, le calcul de chemin, les égalités de poids, l'inatteignabilité et l'erreur poids négatif.
- E7. Documentation : docstrings pour chaque fonction publique et fichier LISEZ-MOI.txt (ou .md).
- E8. Qualité : identifiants explicites (français), séparation logique/entrées-sorties, sous-fonctions, lisibilité.
- E9. Démonstration : fournir ``demonstration.py`` qui charge un CSV, calcule un plus court chemin et affiche la distance et le chemin (voir exemple de sortie).

Entrées / Sorties (format conseillé)

Fichier graphe (CSV, UTF-8), un arc par ligne :

depart,depart_abscisse,depart_ordonnee,arrivee,arrivee_abscisse,arrivee_ordonnee
A,1.2,3.8,B,4.1,5.9
A,1.2,3.8,C,2.4,9.1
C,2.4,9.1,B,4.1,5.9
B,4.1,5.9,D,7.2,3.4
C,2.4,9.1,D,7.2,3.4

La colonne du poids n'étant pas fournie, vous devrez la calculer. Dans ce projet, le **poids d'un arc correspond à la distance euclidienne** entre ses extrémités. Par exemple, le poids de l'arc (A,B) défini précédemment est d'environ 3.58.

Commande type :

```
python main.py --fichier donnees.csv --source A --cible D
```

Sortie attendue :

```
Distance(A->D) = 7.56  
Chemin : A -> B -> D
```

Contraintes

- C1. Travail en binôme.
- C2. Fonctionne tel quel sur Python 3.11 (bibliothèques standards).
- C3. Aucune dépendance externe (pas de networkx, numpy, ...).
- C4. Interdiction d'utiliser une IA générative pour produire du code/tests.
- C5. Respect des bonnes pratiques vues en cours/TD.
- C6. Messages d'erreur/usage clairs et concis (CLI).
- C7. Toute option/format non spécifié explicitement doit être documenté dans le LISEZ-MOI.

Livrables

- L1. Code source : module + main.py + demonstration.py (ou scenario.py).
- L2. Jeux de tests unitaires (unittest ou pytest).
- L3. LISEZ-MOI.txt : instructions d'exécution, format de fichier, réponses aux questions (♦).

Barème (indicatif)

- 1. Dijkstra correct (distances, prédécesseurs, arrêt anticipé, robustesse) — 7 pts
- 2. Conception du module (API FR, invariant, complexité, séparation I/O-logique) — 5 pts

- 3. Tests unitaires (pertinence, cas limites) — 4 pts
- 4. Documentation & LISEZ-MOI — 2 pts
- 5. Qualité de code — 2 pts

Pénalités possibles (indicatif)

- Projet non exécutable (commande fournie échoue) : -5 à -10 pts
- Avertissements/erreurs non justifiés : -1 à -2 pts
- Format d'entrées/sorties non respecté : -1 à -3 pts
- Non-respect de C4 (IA) : note 0

Questions (à traiter dans le LISEZ-MOI)

- Pourquoi Dijkstra requiert-il des poids non négatifs ? Donner un contre-exemple minimal.
- Décrire l'architecture de votre solution (modules, sous-fonctions, structures de données).
- Justifier les structures de données choisies (coûts, mémoire) et l'invariant de représentation.
- Décrivez votre campagne de tests et justifiez la couverture vis-à-vis des cas limites.

Échéances

- Publication du sujet : 11/03/2026
- Rendu final (Moodle) : 12/04/2026